

Linguagem Scala



Var x Val

Em Scala, definimos variáveis usando `var` (mutáveis) e `val` (imutáveis).

- **Val:** Imutável. Uma vez atribuído, o valor não pode ser alterado.
- **Var:** Mutável. Permite a reatribuição de novos valores.

Data Types

- **Int:** Números inteiros.
- **Double / Float:** Números decimais.
- **String / Char:** Textos e caracteres únicos.
- **Boolean:** Valores verdadeiro ou falso.
- **Unit:** Sem valor (equivalente ao *void* em outras linguagens).

Interpolação de Strings

A interpolação de strings permite inserir variáveis diretamente dentro de textos de forma legível. Em Scala, os interpoladores mais comuns são o `s` e o `f`.

1. Interpolador `s` (Simples)

Utilizado para inserir variáveis ou expressões simples. Basta prefixar a string com `s` e usar o cifrão `$` antes do nome da variável.

Exemplo:

```
None  
val nome = "Scala"  
println(s"Olá, $nome!") // Resultado: Olá, Scala!
```

2. Interpolador `f` (Formatado)

Permite formatar números e textos seguindo o estilo *printf*. É ideal para controlar casas decimais e alinhamento.

Exemplo:

```
None  
val preco = 10.5  
println(f"O valor é R$ $preco%.2f") // Resultado: O valor é R$  
10,50
```

Estruturas de Controle (If/Else)

Scala utiliza estruturas condicionais padrão para controle de fluxo.

Match/Case

None

```
variavel match {  
  case 1 => println("Um")  
  case 2 => println("Dois")  
  case _ => println("Outro valor")  
}
```

É similar ao switch/case.

Da para usar if no match/case também

None

```
id match {  
  case x if x > 1 => println("Maior que 1")  
  case x if x < 1 => println("Menor que 1")  
  case _ => println("Qualquer")  
}
```

Laços de Repetição (For e While)

None

```
for (x <- 1 to 10) {  
  println(x)  
}
```

var x: Int = 10

None

```
var x = 10  
while (x >= 0) {  
  println(x)  
  x -= 1  
}
```

O método `.foreach`

O método `foreach` é utilizado para iterar sobre coleções e executar um efeito colateral para cada elemento. Ele não retorna um novo valor (retorna *Unit*).

Exemplo com Map:

None

```
val capitais = Map("Brasil" -> "Brasília", "França" -> "Paris")  
capitais.foreach { case (pais, capital) => println(s"A capital de $pais é $capital") }
```

map e flatMap

map: Transforma cada elemento de uma coleção em um novo elemento, gerando uma nova coleção.

flatMap: Similar ao map, mas "achata" o resultado caso a função de transformação retorne uma coleção.

Exemplo:

None

```
val lista = List(1, 2, 3)
val dobrados = lista.map(_ * 2) // List(2, 4, 6)
val expandidos = lista.flatMap(x => List(x, x * 10)) // List(1, 10, 2, 20, 3, 30)
```

Coleções

- **Seq:** Sequência genérica - Ex: Seq(1031).
- **Set:** Conjunto de elementos únicos sem ordem fixa - Ex: Set(1, 2, 3).
- **Map:** Coleção de pares chave-valor - Ex: Map("chave" -> "valor").
-
- **List:** Lista ordenada de elementos - Ex: List(1, 2, 3, 4, 5).

Principais métodos de List:

filter(condição): Retorna elementos que atendem ao critério.

reduce: Combina elementos em um único valor.

contains(valor): Verifica se o valor existe na lista.

Você pode converter entre elas usando `.toSet` ou `.toList`.

Métodos e Funções

- **Método (def):** Definido dentro de classes ou objetos.

Métodos são funções declaradas dentro de um contexto de objeto ou classe.

Funções (val)

- **Função (val):** Uma função atribuída a uma variável.

Funções podem ser tratadas como valores e atribuídas a variáveis:

None

```
val duplicar = (x: Int) => x * 2
```

Funções de Alta Ordem

São blocos que aceitam funções ou lambdas como argumentos.

Exemplos:

filter(): Filtra elementos com base em um predicado.

map(): Aplica uma transformação a cada elemento.

Orientação a Objetos

Class

Serve como molde para criar objetos usando a palavra-chave `new`.

Estrutura:

None

```
class Pessoa(val nome: String, val idade: Int) {  
    def saudar(): String = s"Olá, meu nome é $nome"  
}
```

Object

Usado para criar instâncias únicas (Singletons). Não requer o uso de `new`.

Exemplo de Calculadora:

None

```
object Calculadora {  
    def soma(a: Double, b: Double): Double = a + b  
    def multiplicacao(a: Double, b: Double): Double = a * b  
}
```

Case Class

Cria uma classe imutável que não exige a palavra-chave `new` para a criação de objetos. Funciona como uma estrutura de dados fixa onde as informações não devem ser alteradas após a criação.

- **Método `.copy()`:** Permite criar uma nova instância alterando apenas atributos específicos, preservando o objeto original.

None

```
case class Usuario(nome: String, idade: Int)
val user1 = Usuario("Junior", 25)
val user2 = user1.copy(idade = 26) // Cria cópia com idade nova
```

Option

Utilizado para representar valores que podem ou não estar presentes, evitando erros de ponteiro nulo (`NullPointerException`).

Possui dois estados possíveis:

- **`Some(valor)`:** Quando o valor existe.
- **`None`:** Quando o valor é ausente.

None

```
val nome: Option[String] = Some("Scala")
val nulo: Option[String] = None
println(nome.getOrElse("Desconhecido")) // Retorna "Scala"
```


Try

Utilizado para tratamento de exceções de forma funcional. Envolve operações que podem falhar (como conversões de tipos ou chamadas de rede).

- **Success(v)**: Operação realizada com sucesso.
- **Failure(e)**: Captura a exceção ocorrida.

None

```
import scala.util.{Try, Success, Failure}
val resultado = Try("123".toInt)
resultado match {
  case Success(v) => println(s"Número: $v")
  case Failure(e) => println(s"Erro: ${e.getMessage}")
}
```

Manipulação de Strings

- **trim**: Remove espaços em branco no início e fim. Exemplo: `texto ".trim`
- **replace**: Substitui caracteres ou trechos. Exemplo: `"1-2".replace("-", ".")`
- **toUpperCase / toLowerCase**: Altera a caixa do texto. Exemplo: `"abc".toUpperCase`

- **split:** Divide a string em um Array baseado em um delimitador.
Exemplo: `"a,b".split(",")`

Referências e Cursos

Scala :

- <https://www.udemy.com/course/apache-spark-with-scala-hands-on-with-big-data/>
- https://www.udemy.com/share/107wNb3@6_025Dq2z7wsGk5A0tQyIKoffLRIH3ErV0A1xTjBqA2gKMbMms0MjEbttM6LtXIzYQ==/